

the machine—run-level 3 will not start with a text login, run-level 5 will start a GUI login, and run-level 1 is single-user, root-only mode for repairing your machine.

You can use '`chkconfig <service name> off`' to prevent a service from running at start-up—<service name> is, of course, the name of the service you want to stop, for example, '`chkconfig sshd off`'.

You can also use '`service -status-all`' to inspect and change the current status of all services listed. You can start a service using '`service <service name> start`', and stop a service using '`service <service name> stop`'. To stop and restart a service, and force it to read and implement any changes you made in its .conf file, use the '`service <service name> restart`' command.

Alternately, if you like the GUI feel, you can use the ntsysv utility to tweak services within the current run-level from its basic cursor interface, which is easier to navigate.

All fired up

Your Linux distro comes with its own firewall—the iptables firewall system. iptables is a 2.4 kernel feature, which was known as ipchains in prior distributions. Chandrasekar's router-and-firewall is based on ipchains, and though he has no

Secured Remote Access

Remote access services are perhaps the easiest way to attack a machine.

Common remote access tools include Telnet, RSH, SSH and FTP. To secure remote access, use the following tips: First, disable access to your computer for Telnet, RSH, and FTP. If you want to keep FTP, at least disable anonymous login. Secured remote access is highly recommended, so make users SSH to your machine and use SCP for transferring files. Both SSH and SCP are built into Red Hat Linux 9. While using SSH, here are some very important tweaks. SSH protocol 1 is

not secure anymore; force the use of protocol 2, which also allows for FTP. Do this by editing `/etc/ssh/sshd_config`. Change 'Protocol 2,1' to 'Protocol 2.' Next, prevent root logins via SSH. Enforce a two-step root login, wherein users first use a low-level account to login, and then `su` to become root. Change '`PermitRootLogin yes`' to '`PermitRootLogin no`'. Finally, don't allow any remote logins without passwords—ensure that the '`PasswordAuthentication yes`' and '`PermitEmptyPasswords no`' parameters remain unchanged.

problems with iptables, he prefers not to fix what ain't broken.

On Red Hat 9, iptables is standard, and we'll show you how to tweak it. Essentially, the firewall system is built into the Linux kernel, and is called Netfilter. iptables is the interface that controls Netfilter, and is relatively easy to configure, once you understand how it works.

As with any firewall system, iptables works with a set of rules; these rules will, for instance, allow all outgoing traffic, block inbound traffic for specific services,

etc. The default firewall (Medium Profile) that you can choose while installing Red Hat Linux 9 is pretty secure—for example, remote access techniques are closed down. This configuration works on the basis of accepting all connections, and then denying specific access. Of course, you can choose to be less, or more, paranoid.

We used the following sample configuration for the firewall on our Linux test rig—sourced from the book *Red Hat Linux 9 for Dummies*. The logic is simple—deny all connections, and then accept for specific services. Each of the following is a command that you can only enter when logged in as root.

You can check your current iptables rules by typing in '`iptables -L`'. Then flush out the current rule set using '`iptables -flush`'.

Next, set the firewall to deny absolutely everything—incoming, outgoing and forwarded traffic—by typing:

```
'iptables -policy INPUT DROP'
'iptables -policy OUTPUT DROP'
'iptables -policy FORWARD DROP'
```

This is ultimate security, but it also means you're cut off from the world. So, we need to start selectively accepting incoming traffic. You need to start out by typing in

```
'iptables -A OUTPUT -j ACCEPT -o lo'
'iptables -A INPUT -j ACCEPT -i lo'
```

These two commands allow the machine's loopback interface to work, thus allowing the OS to send and receive packets that originate from, and are directed to, the machine itself.

```

[root@Gentoo home]# iptables -L
Chain INPUT (policy DROP)
target     port opt source                destination
ACCEPT     all  --  anywhere              anywhere
ACCEPT     all  --  anywhere              anywhere        state NEW,RELATED,ESTABLISHED
ACCEPT     tcp  --  anywhere              anywhere        state NEW,ESTABLISHED
D tcp dpt:ssh
ACCEPT     tcp  --  anywhere              anywhere        state NEW,ESTABLISHED
D tcp dpt:http
ACCEPT     tcp  --  anywhere              anywhere        state NEW,ESTABLISHED
D tcp dpt:sguid
ACCEPT     tcp  --  anywhere              anywhere        state NEW,ESTABLISHED
D tcp dpt:27431
DROP      tcp  --  anywhere              anywhere        tcp dpt:nfs
DROP      udp  --  anywhere              anywhere        udp dpt:nfs
DROP      udp  --  anywhere              anywhere        udp dpt:snmp
DROP      tcp  --  anywhere              anywhere        tcp dpt:snmp
DROP      udp  --  anywhere              anywhere        state NEW,ESTABLISHED
D udp dpt:printer
DROP      tcp  --  anywhere              anywhere        state NEW,ESTABLISHED
D tcp dpt:printer

Chain FORWARD (policy DROP)

```

Use the '`iptables -L`' command to check your setup, after you're done configuring the firewall